TB Futures Product SOP and Design History

Document Control

- Product: TB Futures
- Repository: `TB-vaccine-impact-simulator-`
- Primary document owner: project maintainer
- Last updated: 2026-06-14
- Related documents:
- `README.md`
- `docs/DATA.md`
- `docs/MODEL_CARD.md`

1. Executive Summary

TB Futures is an interactive country-level tuberculosis scenario explorer. It is designed to help users understand how tuberculosis burden may shift under simplified prevention and development scenarios such as higher BCG coverage, improved income level, or a combined intervention context.

The application combines:

- real public-health and economic data
- a machine-learning model trained on historical country-year records
- a FastAPI backend for data access and simulation
- a modern React frontend for presentation-quality exploration

The product is intentionally positioned as a directional exploration tool, not a policy engine, causal inference system, or clinical decision platform.

2. What This Application Does

At a high level, TB Futures lets a user:

1. pick a country
2. choose a scenario or set custom input overrides
3. simulate the model's estimated change in TB incidence
4. view confidence-style uncertainty bounds and narrative explanation
5. inspect a prioritization ranking across countries

6. explore geographic patterns on a world map

7. review model quality and diagnostic context

In practical terms, the app answers questions like:

- If BCG coverage were materially higher, which countries appear to benefit most?
- How different is the model output when economic context improves?
- Which countries combine high TB burden with lower vaccine coverage?
- How reliable is the underlying model, and where are its weaknesses?

3. Why This Was Made

TB Futures was built to solve three connected needs.

3.1 Public-health communication need

Tuberculosis data is often available, but not always easy to explore in a scenario-driven way. Many dashboards show static burden snapshots without letting users test structured "what-if" changes.

3.2 Product demonstration need

The project is also a strong portfolio vehicle because it combines:

- data engineering
- machine learning
- API design
- frontend product thinking
- scientific caveat communication

This makes it more compelling than a static notebook or a one-off model report.

3.3 Decision-support framing need

The app does not claim to prescribe policy. Instead, it helps frame questions:

- where modeled opportunity may exist
- what tradeoffs and uncertainty should be surfaced
- how design can make technical work legible to non-technical viewers

4. Product Positioning

TB Futures should be described as:

- an educational global-health scenario explorer
- a directional prioritization interface

- a presentation-ready health-tech product prototype

TB Futures should not be described as:

- a causal model of vaccine effect
- a policy recommendation engine
- a substitute for epidemiological planning tools
- a clinical platform

5. Intended Users

The current product is most appropriate for:

- recruiters and hiring managers reviewing technical product work
- collaborators evaluating the product direction
- health-tech audiences interested in interpretable scenario tools
- technical reviewers looking at data, model, and UI integration quality

Secondary audiences may include:

- students
- researchers exploring communication concepts
- early product stakeholders assessing future build potential

6. Current System Overview

6.1 Backend

The backend is FastAPI and remains the source of truth for all application data and simulation behavior.

Key endpoints:

- `GET /config`
- `GET /countries`
- `GET /country/{name}`
- `POST /simulate`
- `GET /map-data`
- `GET /whatif-map`
- `GET /prioritize`
- `GET /model-info`

6.2 Data

The product uses public data from OWID, WHO, and World Bank pipelines, merged into a processed country-year modeling dataset.

Core fields include:

- TB incidence
- BCG coverage
- GDP per capita
- population
- income level
- WHO region
- rapid diagnostic site density for contextual display

6.3 Model

The current lead model is a tuned Random Forest regressor trained on transformed TB incidence with comparisons against Linear Regression and Gradient Boosting.

Model framing:

- target is historical country-level TB incidence
- output is directional, not causal
- uncertainty is approximated via forest-tree bootstrap behavior

6.4 Frontend

The new frontend is a Vite React single-page application with:

- React Router
- TanStack Query
- Tailwind CSS
- Recharts
- Plotly
- Framer Motion

Primary routes:

- `/`
- `/prioritization`
- `/map`
- `/model`

7. Design History and Evolution Timeline

This section tracks how the product evolved from a technical prototype into a more intentional product experience.

7.1 Phase 1: Repository setup and framing

Date reference:

- 2026-06-13

Observed history:

- `7cabde9` Initial commit
- `c309b63` README revision and author/project framing

What this phase established:

- the project identity
- the initial repository structure
- the intent to package the work as a coherent product rather than a loose experiment

7.2 Phase 2: Streamlit MVP with API and baseline model

Date reference:

- 2026-06-14

Observed history:

- `7db86ed` Build TB Futures: scenario explorer with FastAPI, Streamlit, and RF model

What changed:

- a working end-to-end experience was created
- FastAPI became the application backend
- Streamlit served as the first user interface
- a Random Forest-based simulation workflow became usable

Why this mattered:

- it validated the core concept quickly
- it proved that users could move from country selection to simulation output in one flow
- it created a demoable artifact early

Constraint discovered:

- Streamlit accelerated delivery but limited visual polish and product feel

7.3 Phase 3: Data and target-quality rework

Date reference:

- 2026-06-14

Observed history:

- `6766fb3` Refactor data pipeline to WHO notifications + income-level feature
- `6441bd5` Make pipeline data-adaptive; WHO-derived TB target

What changed:

- the data pipeline became more robust
- income-level structure was made explicit
- the target moved toward WHO-derived TB burden rather than weaker proxy behavior

Why this mattered:

- the product narrative became more defensible
- the model target became more aligned with what users believe they are exploring
- the app could honestly communicate source provenance

7.4 Phase 4: Tooling and operational hardening

Date reference:

- 2026-06-14

Observed history:

- `220d618` Gitignore generated model metadata JSON
- `088ec85` Make train/evaluate runnable as scripts; fix README run commands

What changed:

- the train/evaluate pipeline became easier to run
- repository hygiene improved
- project onboarding became more straightforward

Why this mattered:

- repeatability improved
- local development friction was reduced

7.5 Phase 5: UI stability fixes

Date reference:

- 2026-06-14

Observed history:

- `0c6cacc` Fix invisible-text bug: force light theme and inline styles

What changed:

- the prototype became more stable for presentation use

Why this mattered:

- presentation blockers undermine trust quickly
- a visible UI bug can outweigh a strong model in stakeholder perception

7.6 Phase 6: React frontend rewrite

Date reference:

- current working tree as of 2026-06-14

What changed:

- a new `frontend/` app was created using React, TypeScript, Vite, and Tailwind
- the user experience moved away from Streamlit as the primary interface
- the visual direction shifted to a warm editorial premium health-tech language
- stateful routes, richer charts, cleaner layout hierarchy, and better motion became possible

Why this mattered:

- the product needed to look intentional and polished, not like a generic data app
- React provides the control necessary for real design-system behavior
- the frontend is now much more suitable for portfolio presentation and stakeholder demos

7.7 Overall design lesson

The history of the application shows a clear pattern:

- fast tools were useful for proving the concept
- stronger data choices made the product more honest
- the final presentation layer required a dedicated frontend stack

That progression is healthy. It means the project matured in the correct order:

1. concept validation
2. data and model tightening
3. product-quality interface refinement

8. Product Experience Principles

The current product direction is based on the following principles.

8.1 Honest intelligence

The app should look strong without overstating certainty. Diagnostics, caveats, and model limitations are part of the product, not an afterthought.

8.2 Calm premium presentation

The UI should feel minimal, warm, spacious, and medically credible rather than loud, over-decorated, or generic enterprise software.

8.3 Dashboard-first clarity

Users should see the key story quickly:

- current burden
- modeled change
- prioritization logic
- methodological confidence

8.4 Scientific restraint

Narrative language should be understandable but should not imply causal proof where only associational modeling exists.

9. Standard Operating Procedure

This section defines how to operate, update, and present the application.

9.1 Local startup procedure

From the repository root:

Command block

```
pip install -r requirements.txt
uvicorn src.api.main:app --reload --port 8000
```

In a second terminal:

Command block

```
cd frontend
cp .env.example .env
npm install
npm run dev
```

Visit:

- frontend: `http://localhost:5173` or the Vite port shown in terminal
- API docs: `http://localhost:8000/docs`

9.2 Data refresh procedure

Use this when upstream source files should be re-synced.

Command block

```
python3 -m src.data.download_data
python3 -m src.data.process_data
```

Expected outcome:

- refreshed raw source files under `data/`
- regenerated processed model dataset

9.3 Model rebuild procedure

Use this when feature logic or data has changed.

Command block

```
python3 -m src.model.train
python3 -m src.model.evaluate
```

Expected checks:

- model artifacts are regenerated
- evaluation metrics complete successfully
- held-out performance does not collapse unexpectedly

9.4 Frontend verification procedure

Command block

```
cd frontend
npm test
npm run build
npm run test:e2e
```

Use the Playwright step when browser binaries are installed. If needed:

Command block

```
npx playwright install
```

9.5 Backend verification procedure

From the repository root:

Command block

```
pytest tests/ -q
```

9.6 Demo procedure

Recommended presentation flow:

1. open homepage and explain the purpose in one sentence
2. choose a recognizable country
3. compare baseline vs combined scenario

4. explain that the result is modeled incidence, not a clinical prediction

5. move to Prioritization and show ranked opportunity

6. move to Map and show spatial context

7. finish on Model page to show methodological transparency

9.7 Operational cautions

- Do not describe output as lives saved unless a separate mortality model exists.
- Do not describe the model as causal.
- Do not use the tool for real policy recommendations without expert review.
- Do not hide poor model performance; contextualize it clearly.

10. What Can Be Added Next

The current product is strong as a polished exploratory demo, but there is substantial room for deeper capability.

10.1 Product features

- saved scenarios and sharable URLs
- downloadable PDF or slide-style country briefs
- side-by-side country comparison
- intervention presets beyond BCG and income context
- onboarding mode for first-time users
- searchable notes or annotations per country

10.2 Scientific and modeling upgrades

- age-band or cohort-specific views
- explicit lag assumptions for vaccine coverage effects
- causal-inference alternatives or quasi-experimental framing where appropriate
- richer health-system features beyond GDP and BCG
- model calibration and uncertainty interval refinement
- scenario constraints grounded in domain literature

10.3 Data upgrades

- automated refresh scheduling
- better version tracking for source data snapshots
- missingness diagnostics in the UI

- population normalization choices exposed to users
- additional burden indicators such as mortality or treatment success

10.4 Experience and design upgrades

- stronger hero media with approved photography or illustration
- narrative scroll mode for guided storytelling
- export-quality charts and branded reports
- usage analytics
- role-based views for technical vs executive audiences

10.5 Engineering upgrades

- deployment pipeline for frontend and backend
- environment-specific config management
- API response caching
- lazy-loaded chart bundles to reduce frontend payload
- background model retraining workflow
- visual regression testing

11. Strategic Potential

The product has value beyond its current form.

11.1 Portfolio potential

TB Futures is already a strong portfolio centerpiece because it demonstrates:

- problem framing
- full-stack execution
- model honesty
- design maturity
- scientific communication

11.2 Product potential

With deeper domain modeling, the app could evolve into a broader intervention planning or opportunity-screening interface for public-health programs.

11.3 Collaboration potential

The architecture is suitable for collaboration across:

- design

- engineering
- data science
- public-health advisory input

11.4 Platform potential

The current architecture could support a family of related products:

- other infectious disease scenario explorers
- health-system capacity dashboards
- intervention prioritization tools
- scientific communication microsites

12. Known Constraints

Current limitations should stay explicit in all presentations and project notes.

- The model learns associations from historical country-level data.
- Inputs are simplified and do not capture transmission dynamics.
- BCG coverage is not a complete description of prevention reality.
- GDP is a crude system-strength proxy.
- Reported uncertainty is approximate.
- Results should be treated as directional exploration only.

13. Recommended Next Phases

If continuing development, the most sensible order is:

1. ship and stabilize the React experience
2. reduce frontend bundle size and improve load performance
3. add exportable country reports
4. improve model diagnostics and uncertainty communication
5. add richer intervention variables
6. establish deployment and monitoring

This order preserves momentum while improving both credibility and usability.

14. Closing Assessment

TB Futures has progressed from a technically useful prototype into a more coherent health-tech product narrative. Its strongest qualities are:

- real-world data grounding
- honest model framing
- clear scenario interaction
- visible design ambition
- strong potential for further extension

Its next challenge is not proving that the concept works. That part is already done. The next challenge is compounding quality:

- sharper performance
- richer scientific inputs
- stronger exports
- more systematic deployment and productization

That is a good place for the project to be.